

PROBABILITÉS

Correction

Exercice 1 : Un jeu du Chevalier de Méré

Ce jeu se joue avec deux joueurs, la banque B et un joueur A :

"Une partie consiste à lancer 4 dés, si aucun six n'est sorti le joueur gagne.

Voici un algorithme pour afficher le gagnant d'une partie. Au lieu de lancer 4 dés, on lance 4 fois un dé et on s'arrête dès qu'un six est sorti."

1. Commenter cet algorithme.

```
//On dispose d'une fonction lancerDe ()
//qui simule le lancer d'un dé
nombre_de_lancers <- 0 #Initialisation de la variable nombre_de_lancers à 0
six=Faux #Initialisation de la variable six à Faux
D <- lancerDe () #Affectation d'un nombre aléatoire entre 1 et 6 à la variable D
Tant que D est différent de 6 et nombre_de_lancers différent de 4 #Mise en place d'une
boucle Tant que
    D <- lancerDe () #Affectation d'un nombre aléatoire entre 1 et 6 à la variable D
    nombre_de_lancers <- nombre_de_lancers + 1 #Comptage du nombre de
lancers
    Si D = 6 #Test de la condition si le dé obtient la face 6
        six=Vrai #Affectation de la valeur Vrai à la variable six
FinTantQue #Fin de la boucle Tant que
Si six = Faux #Test de la condition si aucun 6 n'a été obtenu
    Afficher (le joueur A a gagné) #Affichage du message correspondant
FinDeSi
Sinon #Si au moins 6 a été obtenu
    Afficher (la banque a gagné) #Affichage du message correspondant
FinDeSinon
```

2. Implémenter cet algorithme en langage Python dans une fonction `le_joueurA_gagne ()`.

```
1 from random import*
2 def le_joueur_A_gagne():
3     nombre_de_lancers = 0
4     six=False
5     D = randint(1,6)
6     while D != 6 and nombre_de_lancers != 4:
7         D = randint(1,6)
8         nombre_de_lancers = nombre_de_lancers + 1
9         if D==6:
10             six=True
11     if six == False:
12         return "Le joueur A a gagné."
13     else :
14         return "La banque a gagné."
```

Exercice 2 :

On reprend la situation de l'Exercice 1.

On veut à présent réaliser 5000 parties, par exemple, et calculer la fréquence de gain du joueur A. Pour simplifier le code on va modifier le programme précédent qui retourne deux valeurs VRAI ou FAUX.

1. Faire une fonction permettant de réaliser ces 5000 parties et d'obtenir la fréquence correspondante.

```

16 def frequence_le_joueur_A_gagne():
17     nombre_victoires_joueur_A = 0
18     for x in range(5000):
19         nombre_de_lancers = 0
20         six=False
21         D = randint(1,6)
22         while D != 6 and nombre_de_lancers != 4:
23             D = randint(1,6)
24             nombre_de_lancers = nombre_de_lancers + 1
25             if D==6:
26                 six=True
27             if six == False:
28                 nombre_victoires_joueur_A+=1
29     frequence=nombre_victoires_joueur_A/5000
30     return 'La fréquence de parties gagnées par le joueur A est de '+str(frequence)+'.'

```

2. Qui a le plus de chance de gagner à ce jeu ?

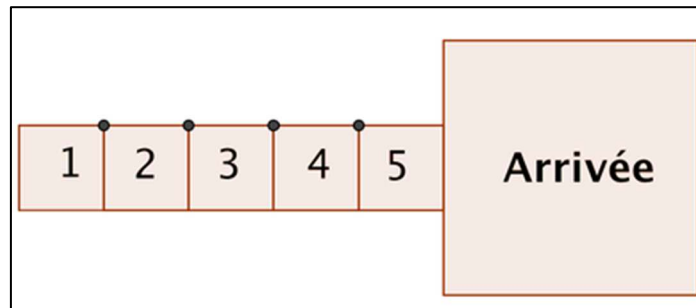
La fréquence de gain du joueur A étant plus grande que 0,5. Il a plus de chance de gagner à ce jeu.

3. Quel est la fréquence moyenne approximative de gain du joueur A ?

Le joueur A perd la partie dans le cas où il obtient les faces 1, 2, 3, 4 ou 5 aux quatre lancers de dé. La probabilité que le joueur A perde la partie est donc de $\left(\frac{5}{6}\right)^4$.

Ainsi la fréquence de gain du joueur A est de $1 - \left(\frac{5}{6}\right)^4$, c'est-à-dire environ 0,518.

Exercice 3 : Course aléatoire



L (le lièvre) et T (la tortue) sont au départ d'une course. L'arrivée est six cases plus loin. Tant que l'un des deux joueurs n'est pas arrivé, on lance un dé.

Si le résultat est six le lièvre va immédiatement sur la case arrivée, sinon la tortue avance d'une case.

1. Rédiger un algorithme pour simuler une partie.

```

Nombre_lancers_de <- 0
De <- 0
Tant que De < 6 et Nombre_lancers_de < 6
    De <- lancerDe ()
    Nombre_lancers_de <- Nombre_lancers_de + 1
Si Nombre_lancers_de = 6
    Afficher ("La tortue gagne la partie")
Sinon
    Afficher ("Le lièvre gagne la partie")

```

2. Traduire cet algorithme en Python et le tester.

```
32 nombre_de_lancers_de = 0
33 de = 0
34 while de < 6 and nombre_de_lancers_de < 6:
35     de = randint(1,6)
36     nombre_de_lancers_de = nombre_de_lancers_de + 1
37 if nombre_de_lancers_de == 6:
38     print ("La tortue gagne la partie")
39 else:
40     print ("Le lièvre gagne la partie")
```

3. Rédiger un algorithme pour réaliser 1000 parties, puis calculer les fréquences pour savoir lequel des deux animaux a gagné le plus de parties.

```
Fonction le_lièvre_et_la_tortue()
Victoires_tortue <- 0
Pour 1000 parties
    Nombre_lancers_de <- 0
    De <- 0
    Tant que De < 6 et Nombre_lancers_de < 6
        De <- lancerDe ()
        Nombre_lancers_de <- Nombre_lancers_de + 1
    Si Nombre_lancers_de = 6
        Victoires_tortue <- Victoires_tortue + 1
Renvoyer Victoires_tortue/1000 et 1 - Victoires_tortue/1000
```

4. Traduire cet algorithme en Python.

```
42 def le_lièvre_et_la_tortue():
43     victoires_tortue=0
44     for x in range(1000):
45         nombre_de_lancers_de = 0
46         de = 0
47         while de < 6 and nombre_de_lancers_de < 6:
48             de = randint(1,6)
49             nombre_de_lancers_de = nombre_de_lancers_de + 1
50         if nombre_de_lancers_de == 6:
51             victoires_tortue+=1
52     frequence_tortue=victoires_tortue/1000
53     return 'La fréquence de parties gagnées par la tortue est de '+str(round(frequence_tortue,3))+'. La fréquence de parties gagnées par le lièvre est de '+str(round(1-frequence_tortue,3))+'
```